

Day 9 Task Report

Title: AI Pipeline Design

TASK-09: Engineering a Robust Input-Output Pipeline for RAG

Objective

The objective of this task is to build a **production-level RAG pipeline** that is not only functional but also robust and secure.

Unlike the previous "Happy Path" system, this pipeline is designed to:

- Protect the system from invalid or harmful inputs
 - Track system performance using latency logging
 - Handle API failures gracefully without crashing
-

Tools Used

- VS Code
 - Git & GitHub
 - MS Word
-

Libraries Used

- **PyPDF2** : To read and extract text from PDF
 - **google-genai** : To generate responses using Gemini
 - **dotenv** : To securely manage API keys
 - **time** : To calculate latency
 - **re** : To perform input validation
-

Concepts Used

1. Input Validation

Ensures that only meaningful and safe user queries are processed.

2. Latency Logging

Measures system performance by tracking:

- Retrieval time
- Generation time

3. Graceful Failure Handling

Prevents system crashes by handling API errors properly.

4. Retrieval-Augmented Generation (RAG)

Uses PDF data as context to generate accurate responses.

Code Explanation

The system works in the following steps:

1. Load API key using dotenv
2. Read the PDF file using PyPDF2
3. Extract text from all pages
4. Split text into smaller chunks
5. Take user input dynamically
6. Validate user input
 - Reject queries longer than 300 characters
 - Reject empty or meaningless inputs
7. Retrieve the most relevant chunk from the PDF
8. Send context + query to Gemini model
9. Generate response based on context

10.Measure:

- Retrieval time
- Generation time

11.Store performance in performance.log

12.Handle API failure using try-except

Errors Faced

1. PDF Reading Error

Problem:

System crashed while reading PDF. Some pages returned None instead of text.

Solution:

Handled safely:

```
text += page.extract_text() or ""
```

2. Function Call Mistake

Problem:

Error occurred:

`TypeError: run_pipeline() missing 1 required positional argument`

Solution:

Corrected function call:

```
run_pipeline(query, chunks)
```

3. Special Characters Issue

Problem:

Inputs like `!!! ???` were meaningless but still processed.

Solution:

Used regex to ensure input contains valid characters.

Output

```
(.venv) PS D:\Qualtech\Daily Tasks\Day9> py -3.10 pipeline.py
Loading PDF...
Chunking text...
System Ready! Ask your questions
```

Enter your query (or type 'exit'): what is toknization ?

Response:

Service temporarily unavailable. Please try again in 30 seconds.

Enter your query (or type 'exit'): █

Enter your query (or type 'exit'): what is pluse core evolution ?

Response:

Based on the provided text:

***The Pulse -Core Evolution** is presented as the **title of section 1.1** within the document about the CORE KERNEL ARCHITECTURE & TEMPORAL SHARDING.

The text itself doesn't explicitly define "Pulse-Core Evolution" as a *process*. Instead, it introduces the ***Pulse -Core*** as a key architectural component of the NexusVortex Operating System.

What we learn about the "Pulse-Core" (which "Pulse-Core Evolution" likely refers to the development or understanding of):

- * It is a core component of the **NexusVortex Operating System**.
- * It represents a ***paradigm shift in decentralized computing.***
- * It is distinct from and an alternative to ***traditional monolithic kernels*** found in legacy systems like Windows, macOS, or Linux.

In essence, "Pulse-Core Evolution" is the heading for the section that introduces and explains this new, non-monolithic kernel architecture used by NexusVortex. The text *starts* to tell us *what* the "Pulse-Core" is by contrasting it with traditional kernels.